

# 开源软件基础

## 概念与历史

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 7, 2017



# Outline

- 1 关于我
- 2 特色栏目
- 3 教学内容
- 4 定义

# 关于我

## 任志磊

- 副教授
- 2003 年进入软件学院学习
- <http://oscar-lab.org/people/~zren>
- [zren@dlut.edu.cn](mailto:zren@dlut.edu.cn)
- 研究兴趣:
  - 进化计算
  - 数据挖掘
  - 软件工程应用
- <https://bitbucket.org/rezilla/open-lecture-notes.git>

# 所在团队

## OSCAR

- 江贺教授团队
- <http://oscar-lab.org>
- “**O**ptimizing **S**oftware by **C**omputation from **AR**tificial intelligence”
- or “**O**perating **S**ystem will **C**rash whenever my **Al**gorithm **R**uns”

# 所在团队

## OSCAR

- 江贺教授团队
- <http://oscar-lab.org>
- “**O**ptimizing **S**oftware by **C**omputation from **AR**tificial intelligence”
- or “**O**perating **S**ystem will **C**rash whenever my **AL**gorithm **R**uns”

# 关于课程

## 开源的几个含义

- 课程主题为开源软件
- 课程使用软件均为开源软件
- 课件开源发布

# Outline

- 1 关于我
- 2 特色栏目
- 3 教学内容
- 4 定义

# 特色栏目

- 人物介绍
- 今天就能开始用的小知识
- 书籍推荐



## 人物介绍

Richard Stallman<sup>a</sup>

<sup>a</sup><https://zh.wikipedia.org/zh-cn/û>

美国程序员，自由软件活动家。他发起自由软件运动，倡导软件使用者能够对软件自由地进行使用、学习、共享和修改，确保了这些软件被称作自由软件。斯托曼发起了 GNU 项目，并成立了自由软件基金会。他开发了 GCC、GDB、GNU Emacs，同时编写了 GNU 通用公共许可协议。



# 今天就能开始用的小知识

## 关于 Richard Stallman 的事实

- Richard Stallman 很长时间使用一台龙芯笔记本
- Richard Stallman 的键盘是 HHKB



# 书籍推荐



# Outline

- 1 关于我
- 2 特色栏目
- 3 教学内容**
- 4 定义

# 教学大纲

- 开源软件的概念和历史，开源软件仓库与开源软件生态系统
- 开源软件许可证制度，开源软件成熟度评估与选型
- 开源软件仓库结构
- 基于 python 的数据获取 ( 1 ): python 入门

# 教学大纲

- 基于 python 的数据获取 ( 2 ): python 的进阶知识
- 基于 python 的数据获取 ( 3 ): requests、scrapy 与 pyspider
- 基于 python 的数据获取 ( 4 ): 基于 BeautifulSoup 的数据过滤
- 版本控制的概念与入门

# 教学大纲

- 基于 git 的团队协作
- 基于 nose 的测试自动化
- git 提交日志分析
- 基于 sqlalchemy 的结构化数据存储

# 教学大纲

- 基于 pymongoodb 的半结构化数据存储
- 基于 kettle 的数据转换
- 基于 pandas 的开源软件数据探索性分析
- 基于 matplotlib 和 ggplot 的图表生成自动化
- 基于 webpy 与 django 的 web 应用框架



# Outline

- 1 关于我
- 2 特色栏目
- 3 教学内容
- 4 定义**

# 开源软件的定义

## 定义

开放源代码的定义由 Bruce Perens ( Debian 的创始人之一 ) 定义如下:

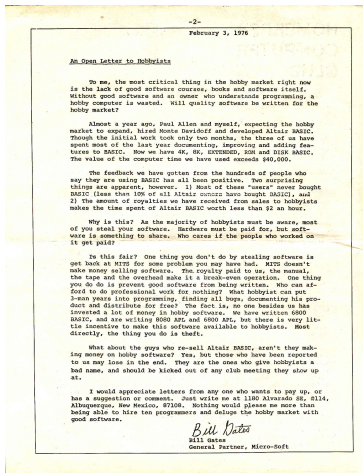
- 自由再散布 ( Free Distribution ): 允许获得源代码的人可自由再将此源代码散布。
- 源代码 ( Source Code ): 程序的可执行文件在散布时, 必需以随附完整源代码或是可让人方便的事后获取源代码。
- 派生著作 ( Derived Works ): 让人可依此源代码修改后, 在依照同一许可协议的情形下再散布。
- 原创作者程序源代码的完整性 ( Integrity of The Author's Source Code ): 意即修改后的版本, 需以不同的版本号码以与原始的代码做分别, 保障原始的代码完整性。
- 不得对任何人或团体有差别待遇 ( No Discrimination Against Persons or Groups ): 开放源代码软件不得因性别、团体、国家、族群等设置限制, 但若是因为法律规定的情形则为例外 ( 如: 美国政府限制高加密软件的出口 )。

# 开源软件的定义（续）

## 定义

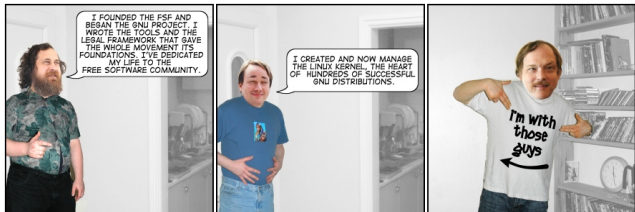
- 对程序在任何领域内的利用不得有差别待遇 ( No Discrimination Against Fields of Endeavor ): 意即不得限制商业使用。
- 散布许可协议 ( Distribution of License ): 若软件再散布, 必需以同一条款散布之。
- 许可协议不得专属于特定产品 ( License Must Not Be Specific to a Product ): 若多个程序组合成一套软件, 则当某一开放源代码的程序单独散布时, 也必需要匹配开放源代码的条件。
- 许可协议不得限制其他软件 ( License Must Not Restrict Other Software ): 当某一开放源代码软件与其他非开放源代码软件一起散布时 ( 例如放在同一光盘 ), 不得限制其他软件的授权条件也要遵照开放源代码的授权。
- 许可协议必须技术中立 ( License Must Be Technology-Neutral ): 意即许可协议不得限制为电子格式才有效, 若是纸本的许可协议也应视为有效。

# An open letter to hobbyists<sup>1</sup>



# 开源软件的历史

1997 年, Eric Raymond 出版其著作《大教堂和市集》, 探讨黑客社区与自由软件原则。1998 年初, 该论文受到极大的关注, 为促成网景通信公司将其受欢迎的互联网套装软件 Netscape 释放成为自由软件的因素之一。这些代码即为今日大家熟悉的 Mozilla Firefox 与 Thunderbird。网景的行动激起 Raymond 及其伙伴深入研究如何将自由软件基金会的自由软件概念及优点带入商业软件产业。他们查觉基金会的社会活动不如网景等公司的行动来得吸引人, 因而试图重新包装自由软件运动, 以强调分享与协作软件源代码的潜在商机。他们选用的新名称为“开放源代码”(open source), 很快地布鲁斯·佩伦斯、出版家提姆·奥莱理、林纳斯·托瓦兹及其他人支持新名称。开放源代码促进会于 1998 年 2 月创建, 以推动使用新名称, 并宣扬开放源代码的原则。



# 向前追溯

## UNIX

Unix 的前身为 1964 年开始的 Multics，贝尔实验室（Bell Labs）1965 年时，加入一项由通用电气（General Electric）和麻省理工学院（MIT）合作的计划；该计划要创建一套多用户、多任务、多层次（multi-user、multi-processor、multi-level）的 MULTICS 操作系统。贝尔实验室参与了这个操作系统的研发，但因为开发速度太慢，1969 年贝尔实验室决定退出这个计划。贝尔实验室的工程师，肯·汤普森和丹尼斯·里奇，在此时自行开发了 Unix。

# 向前追溯

## UNIX

此后的 10 年，Unix 在学术机构和大型企业中得到了广泛的应用，当时的 UNIX 拥有者 AT&T 公司以低廉甚至免费的许可将 Unix 源码授权给学术机构做研究或教学之用，许多机构在此源码基础上加以扩充和改进，形成了所谓的"Unix 变种"，这些变种反过来也促进了 Unix 的发展，其中最著名的变种之一是由加州大学柏克莱分校开发的柏克莱软件包 (BSD) 产品。

# 向前追溯

## UNIX

后来 AT&T 意识到了 Unix 的商业价值，不再将 Unix 源码授权给学术机构，并对之前的 Unix 及其变种声明了版权权利。BSD 在 Unix 的历史发展中具有相当大的影响力，被很多商业厂家采用，成为很多商用 Unix 的基础。其不断增大的影响力终于引起了 AT&T 的关注，于是开始了一场持久的版权官司，这场官司一直打到 AT&T 将自己的 Unix 系统实验室卖掉，新接手的 Novell 采取了一种比较开明的做法，允许柏克莱分校自由发布自己的 Unix 变种，但是前提是必须将来自于 AT&T 的代码完全删除，于是诞生了 4.4 BSD Lite 版，由于这个版本不存在法律问题，4.4 BSD Lite 成为了现代柏克莱软件包的基础版本。尽管后来，非商业版的 Unix 系统又经过了很多演变，但其中有不少最终都是创建在 BSD 版本上（Linux、Minix 等系统除外）。



# 向前追溯

## UNIX vs. GNU vs. Linux

- 1984 年, Richard Stallman 发起了 GNU 项目, 目标是创建一个完全自由且向下兼容 UNIX 的操作系统。这个项目不断发展壮大, 包含了越来越多的内容。现在, GNU 项目的产品, 如 Emacs、GCC 等已经成为各种其它自由发布的类 UNIX 系统中的核心角色。
- 1990 年, Linus Torvalds 决定编写一个自己的 Minix 内核, 初名为 Linus' Minix, 意为 Linus 的 Minix 内核, 后来改名为 Linux。此内核于 1991 年正式发布, 并逐渐引起人们的注意。当时 GNU 操作系统仍未完成, GNU 系统软件集与 Linux 内核结合后, GNU 软件构成了这个 POSIX 兼容操作系统 GNU/Linux 的基础。今天 GNU/Linux 已经成为发展最为活跃的自由 / 开放源码的类 Unix 操作系统。
- 1994 年, 受到 GNU 工程的鼓舞, BSD 走上了复兴的道路。BSD 的开发也走向了几个不同的方向, 并最终导致了 FreeBSD、NetBSD、OpenBSD 和 DragonFlyBSD 等基于 BSD 的操作系统出现。
- 2007 年发布的 Mac OS X 10.5 Leopard 通过 The Open Group 的 UNIX03 认证。

# UNIX

## 今天就能开始用的小知识

- Minix 是处于教学目的的一个类 Unix 克隆，作者是 Andrew S. Tanenbaum
- UNIX 是一个注册注册商标
- 目前它的商标权由国际开放标准组织所拥有，只有匹配单一 UNIX 规范的 UNIX 系统才能使用 UNIX 这个名称，否则只能称为类 UNIX (UNIX-like)
- UNIX 哲学：做一件事并做好；让程序能够互相协同工作；让程序处理文本数据流

# 相关定义

## 自由软件

根据 Richard Stallman 和自由软件基金会 (FSF) 的定义, 自由软件赋予使用者四种自由:

- 自由之零: 不论目的为何, 有使用该软件的自由。
- 自由之一: 有研究该软件如何运作的自由, 并且得以修改该软件来符合使用者自身的需求。取得该软件之源码为达成此目的之前提。
- 自由之二: 有重新散布该软件的自由, 所以每个人都可以藉由散布自由软件来敦亲睦邻。
- 自由之三: 有改善再利用该软件的自由, 并且可以发表修訂後的版本供公众使用, 如此一来, 整个社群都可以受惠。如前项, 取得该软件之源码为达成此目的之前提。

如果一软件的使用者具有上述四种权利, 则该软件得以被称之为「自由软件」。也就是说, 使用者必须能够自由地、以不收费或是收取合理的散布费用的方式、在任何时间再散布该软件的原版或是改写版, 在任何地方给任何人使用。如果使用者不必问任何人或是支付任何的许可费用从事这些行为, 就表示她 / 他拥有自由软件所赋予的自由权利。

Free as in free speech, not as in free beer.

# 今天就能开始用的小知识：免费啤酒许可证<sup>2</sup>

## Beer-ware License

```
/*  
* _____  
* "THE BEER-WARE LICENSE" (Revision 42):  
* <phk@FreeBSD.ORG> wrote this file. As long as you retain this notice you  
* can do whatever you want with this stuff. If we meet some day, and you think  
* this stuff is worth it, you can buy me a beer in return. Poul-Henning Kamp  
* _____  
*/
```

<sup>2</sup><https://people.freebsd.org/~phk/>

# 相关定义

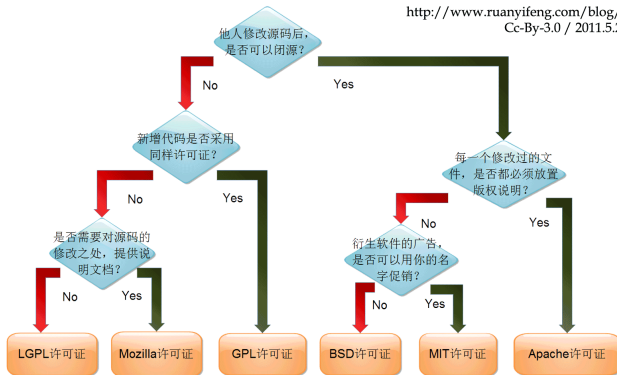
## 自由软件 vs. 开源软件

开放源代码的规定较为宽松，而自由软件的规定更加有严格。很多的开放源代码所认可的授权不算自由软件。

- 开放源代码作用是，使用开放的开发方式，尽可能的使软件最优化，而自由软件则将尊重用户自由作为道德标准。
- 如果说“自由软件”会引起误解，（因为英文“Free”一词有“自由”、“免费”的双重含意），那么“开放源代码”的名字则会引起的误解则更多。“开源”很容易让人认为是只要把源代码“公开”出来就算是开源了，即“你可以看到源代码”。但是如果用户的自由仍然得不到尊重，那么即使公开源代码也没有意义。有的软件公司只是为了想找用户帮它除错、吸收社区贡献的功能，这样子会破坏了自由软件的原意。一个例子是 Tivo 公司生产的机顶盒。虽然它基于 GNU/Linux，TiVo 公司也按照许可证发布了源代码，但是却禁止用户在机顶盒上运行自己的程序，或重新安装系统。[9]
- 自由软件的原意就是要给予用户运用软件的自由，这个‘自由’就是自由软件的精神所在。但是商业化开放的源代码却故意忽略了这个最重要的精神，反而无法让用户体认到‘自由’的真意，那么开源这一个替代自由软件的辞句反而把自由的原意除去了。

# 许可证选择

<http://www.ruanyifeng.com/blog/Cc-BY-3.0/> / 2011.5.2



# Linux 与 GPL

“I used to be worried about fragmentation, and I used to think that it was inevitable at some point,” said Torvalds. “Everyone was looking at the history of Linux and comparing it with UNIX. People would say that it’s going to fail because it’s going to fragment. That’s what happened before, so why even bother?”

What made the difference was the license. “FSF [Free Software Foundation] and I don’t have a loving relationship, but I love GPL v2,” said Torvalds. “I really think the license has been one of the defining factors in the success of Linux because it enforced that you have to give back, which meant that the fragmentation has never been something that has been viable from a technical standpoint.”<sup>3</sup>

---

<sup>3</sup><https://www.infoworld.com/article/3112778>

# Linux 与 GPL

“我过去担心在特定时间 Linux 会分裂，”Torvalds 说，“每个回顾 Linux 历史的人都会将其与 UNIX 进行比较。人们说既然会分裂则一定会失败。过去就发生过，为什么还要去做？”

使情况变化的是许可证协议。“我对自由软件基金会并不感冒，但是我喜爱 GPL v2，”Torvalds 说。“我确实认为许可证是保证 Linux 成功的决定性因素，因为它确保你需要在获取的同时给予回报，这意味着分裂从技术角度就是不会发生的。”<sup>4</sup>

---

<sup>4</sup><https://www.infoworld.com/article/3112778>



# 开源软件仓库

## 定义

软件仓库是收藏互联网上可用软件包 (应用程序) 的"图书馆"。通过软件仓库用户可以获取和安装软件包。开源软件仓库是针对开源软件的软件仓库。

大多数软件仓库将仓库部署在互联网服务器上。软件仓库可能针对特定语言编写的程序, 如 Perl 语言的 CPAN 仓库或 Python 的 PyPI, 或者针对整个操作系统, 如大多数 Linux 发行版。大多数软件仓库为用户提供包管理器 (Package Manager), 用来解决软件包搜索、安装、卸载、处理软件包之间依赖关系。例如 Debian 及其衍生发行版 (如 Ubuntu) 提供的 APT (Advanced Packaging Tool)。

# 开源软件仓库

|                                                  |         |
|--------------------------------------------------|---------|
| C++                                              | Boost   |
| Haskell                                          | Hackage |
| Java                                             | Maven   |
| .NET                                             | NuGet   |
| Node.js                                          | NPM     |
| Perl                                             | CPAN    |
| PHP                                              | PEAR    |
| Python                                           | PyPI    |
| R                                                | CRAN    |
| T <sub>E</sub> X/L <sub>A</sub> T <sub>E</sub> X | CTAN    |

C[PRT]AN 代表 Comprehensive Perl/R/T<sub>E</sub>X Archive Network

# 开源软件仓库

|                                                  |         |
|--------------------------------------------------|---------|
| C++                                              | Boost   |
| Haskell                                          | Hackage |
| Java                                             | Maven   |
| .NET                                             | NuGet   |
| Node.js                                          | NPM     |
| Perl                                             | CPAN    |
| PHP                                              | PEAR    |
| Python                                           | PyPI    |
| R                                                | CRAN    |
| T <sub>E</sub> X/L <sub>A</sub> T <sub>E</sub> X | CTAN    |

C[PR]TAN 代表 Comprehensive Perl/R/T<sub>E</sub>X Archive Network

# 开源生态系统

软件生态系统值一组在相同环境下共同开发和协同进化的一组软件系统。生态系统可以指组织（软件公司），社会（开源社区）或技术（如 Python 技术栈）。

软件生态系统词组使用较为广泛，不过尚存在争议<sup>5</sup>。本课程中取技术角度的含义。

---

<sup>5</sup><https://www.gnu.org/philosophy/words-to-avoid.html#Ecosystem>

# 主力语言

- C/C++: Gcc/llvm
- Java
- C#: via mono
- Rust: 讨论过程开源: <https://github.com/rust-lang/rfcs>

# 胶水语言

- Python
- Perl
- PHP
- Ruby
- Bash
- PowerShell
- JavaScript
- Lua
- R
- Julia

# 数据库

- Oracle
- MySQL
- Postgresql
- MongoDB
- Redis
- Sqlite

# 服务器

- IIS
- nginx
- Apache

Table 1: 市场份额, 2017/10<sup>6</sup>

|                      |                        |       |
|----------------------|------------------------|-------|
| Apache               | Apache                 | 48.5% |
| nginx                | NGINX, Inc.            | 35.4% |
| IIS                  | Microsoft              | 10.8% |
| LiteSpeed Web Server | LiteSpeed Technologies | 2.9%  |
| GWS                  | Google                 | 1.1%  |

<sup>6</sup>[https://en.wikipedia.org/wiki/Web\\_server](https://en.wikipedia.org/wiki/Web_server)



# Web 框架

- J2EE
- Dancer
- Django
- Ruby on Rails
- Zend

# 数据挖掘/机器学习

- Weka
- CRAN
- sk-learn
- tensorflow
- torch

# 前端

- Angular
- Express
- Ext.js
- Bootstrap
- Vue.js

# GUI

- Qt
- Gtk
- MFC
- WPF
- wxWidgets

# 软件测试

- cpptest
- Googletest
- Junit
- nose
- pytest

# 分布式系统

- Hadoop
- Spark
- Storm
- Openstack

# 虚拟化

- Openstack
- Docker
- debootstrap
- libvirt

# 数据获取

- nutch
- scrapy
- requests



# 版本控制

- SVN
- Git
- Mercurial
- Bazaar



# 可视化

- gnuplot
- matplotlib
- ggplot2

# 文档

- $\text{\LaTeX}$
- MS Office
- Doxygen
- Wiki
- sphinx

# 其他

## Functional programming

- Lisp
- Haskell
- Ocaml
- R

# 其他

## Literate Programming

文学编程是由高德纳提出的编程方法，希望能用来取代结构化编程范型。文学编程范型不同于传统的由计算机强加的编写程序的方式和顺序，而代之以让程序员用他们自己思维内在的逻辑和流程所要求的顺序开发程序。

- cweb
- knitr
- jupyter

# 其他

## 编辑器/开发环境

- vim
- emacs
- sublime
- atom
- vscode